Toward Multi-SLO-Aware Scheduling for Heterogeneous AI Accelerators on Edge Devices

Agenda

- Motivation and Research Challenges.
- Proposed Approach
  1. Research Part #1 – Multi-SLO heterogeneous scheduling
  2. Research Part #2 – Model-aware dynamic inference adaptation
  3. Research Part #3 – Elastic model and runtime co-design
  4. Evaluation Plan
- Timeline, Summary, and Contributions

Agenda

- **Motivation and Research Challenges.**
- Proposed Approach
  1. Research Part #1 – Multi-SLO heterogeneous scheduling
  2. Research Part #2 – Model-aware dynamic inference adaptation
  3. Research Part #3 – Elastic model and runtime co-design
  4. Evaluation Plan
- Timeline, Summary, and Contributions

Research Motivation

- Edge devices are resource-constrained (limited power, memory, compute) yet must run complex AI models
- Applications demand multiple concurrent guarantees: high accuracy AND low latency AND energy efficiency
- Current systems waste 40-60% of computational resources by executing full models even when unnecessary
- Existing schedulers achieve <60% multi-SLO satisfaction rates, failing to meet user requirements

Research Motivation (Needs simplification)

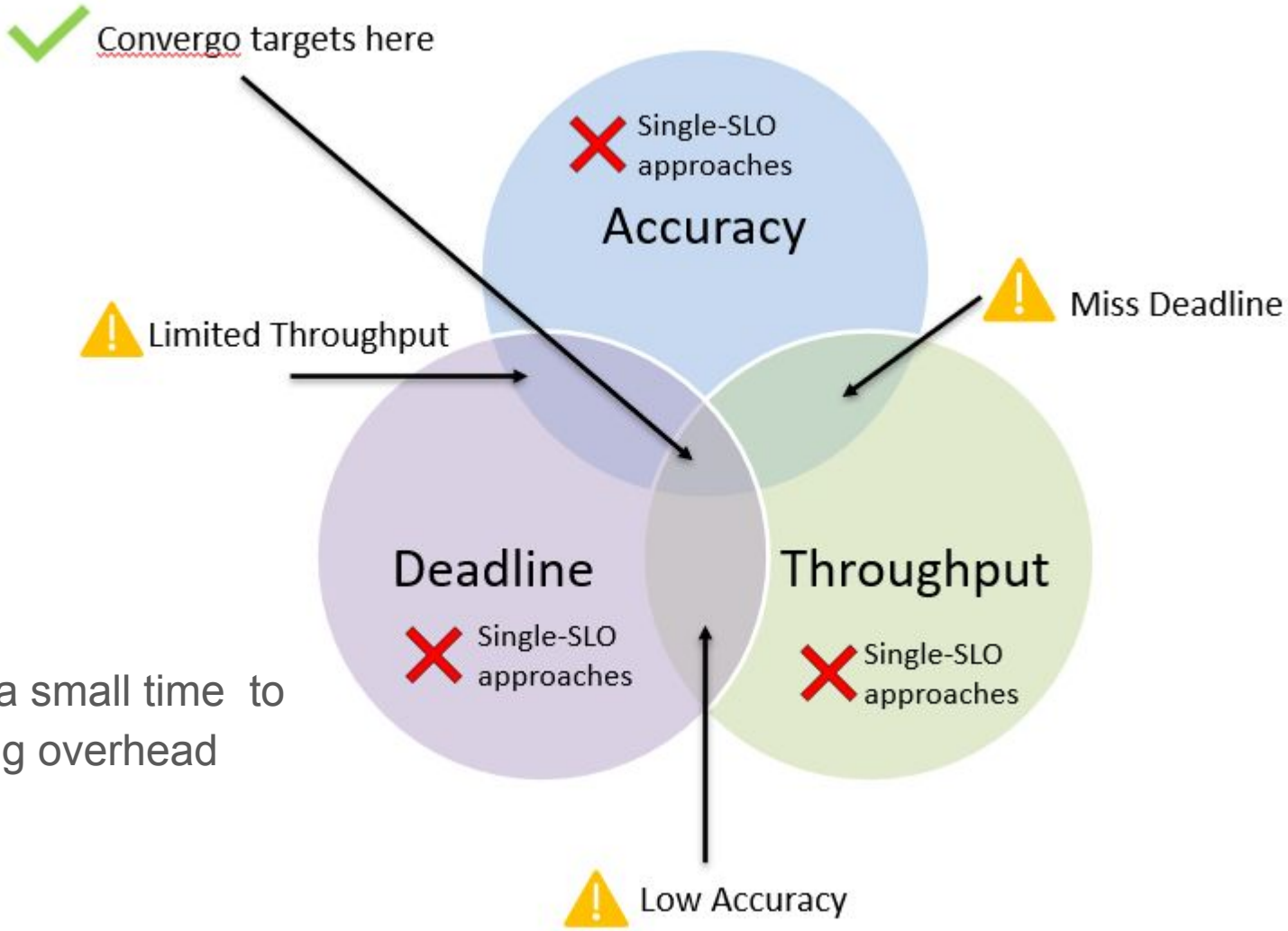
- Edge devices are resource-constrained (limited power, memory, compute) yet must run complex AI models
- Applications demand multiple concurrent guarantees: high accuracy AND low latency AND energy efficiency
- Current systems waste 40-60% of computational resources by executing full models even when unnecessary
- Existing schedulers achieve <60% multi-SLO satisfaction rates, failing to meet user requirements

Research Challenges

Current IoT-edge Systems:

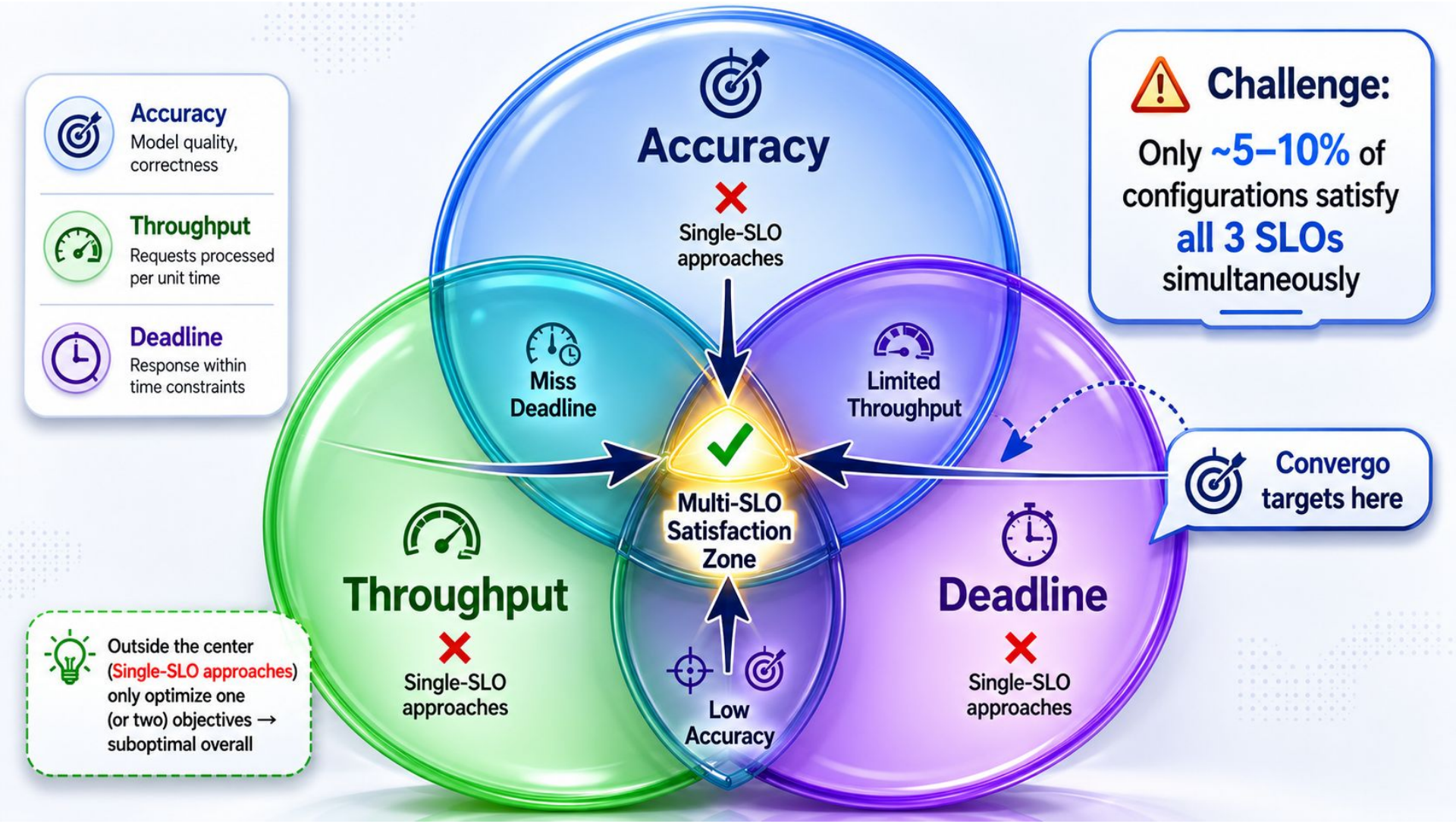
- **Multi-SLO Heterogeneous Scheduling**
- **Model-Aware Dynamic Inference Adaptation**
- **Elastic Model and Runtime Co-Design**

Challenge #1:
Multi-SLO Heterogeneous Scheduling



- Must decide in a small time to avoid scheduling overhead

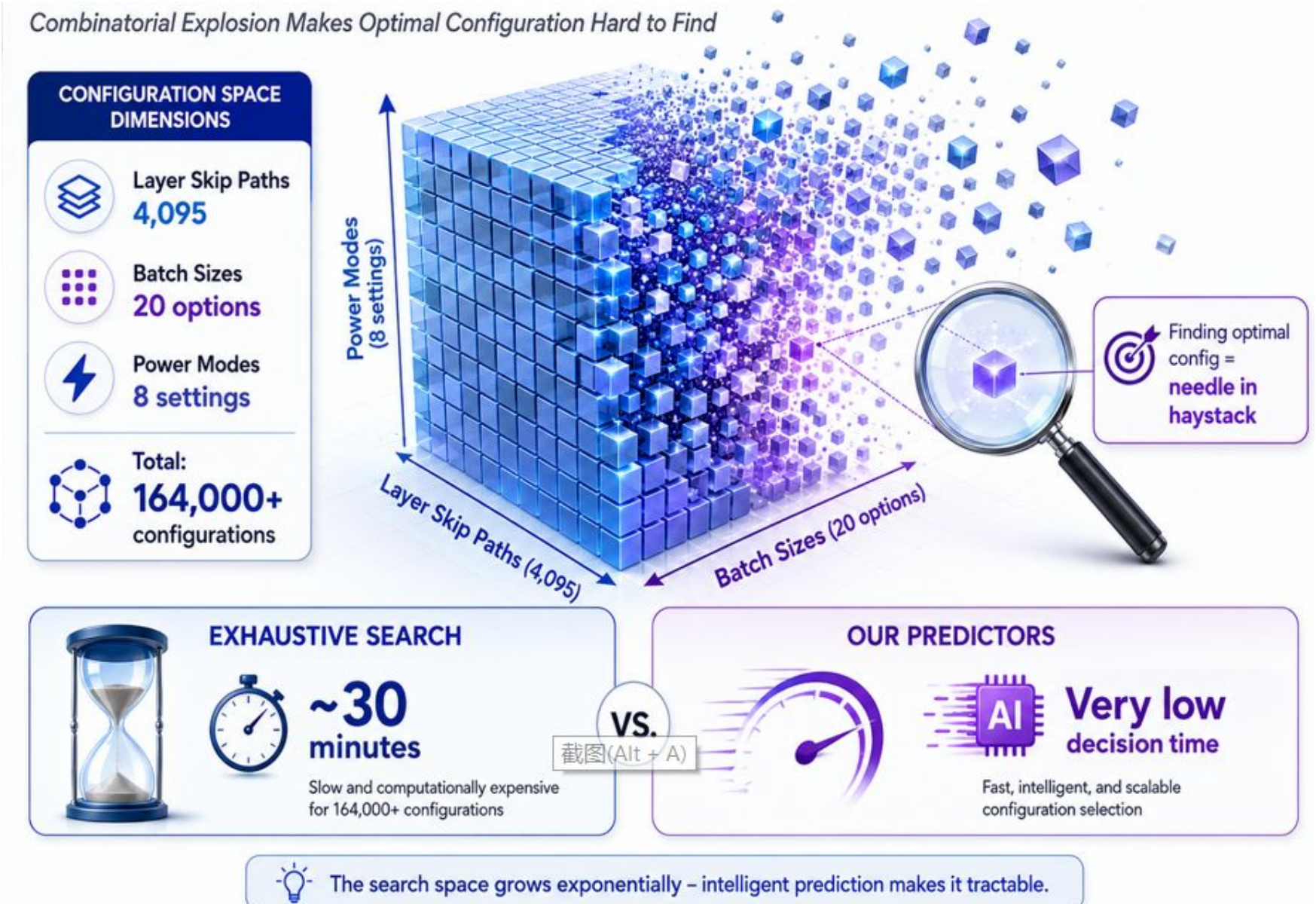
Challenge #1:
Multi-SLO Heterogeneous Scheduling



Resource-constraints

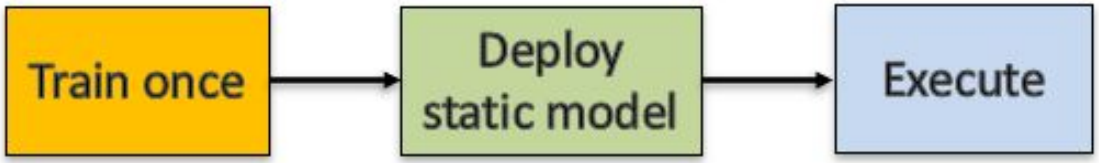
Device	CPU	GPU	Memory	Power	Price
Raspberry Pi 4B	Quad-core Cortex-A72 @1.5GHz	N/A	4GB	**1.8-5.3W**	**$55**
Odroid N2	Quad-core Cortex A73 Dual-core Cortex-A53 @1.8GHz	Mali-G52	4GB	**2-5W**	$79
Jetson Nano	Quad-core Cortex-A57 @1.5GHz	128-core NVIDIA Maxwell @1.0GHz	4GB	5/10W	$99
Jetson TX2	Quad-core Cortex-A73 Dual-core Cortex-A53 @**2.0GHz**	**256-core** NVIDIA Pascal @**1.3GHz**	**8GB**	7.5/15W	$399

Challenge #2:
Model-Aware Dynamic Inference Adaptation

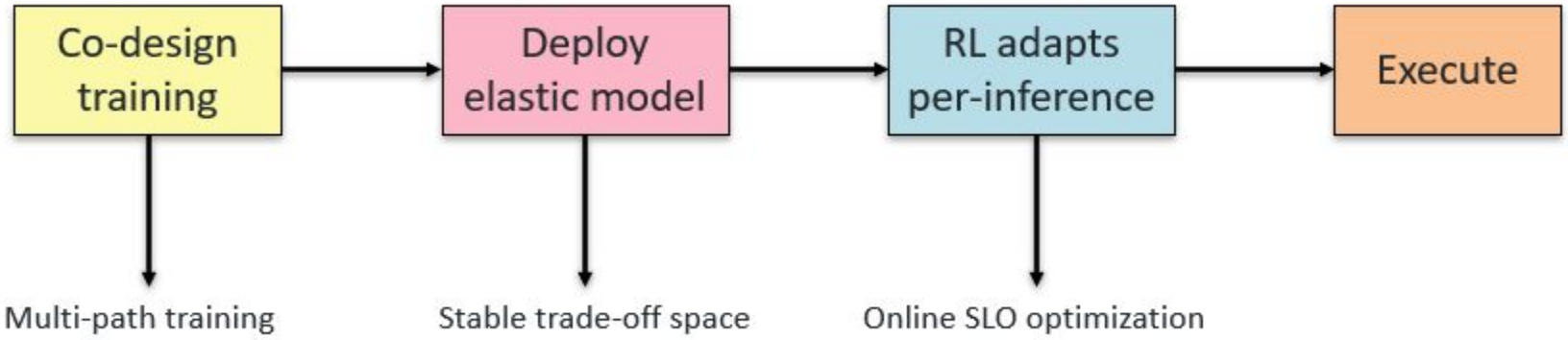


Challenge #3:
Elastic Model and Runtime Co-Design

- **Prior Work:**



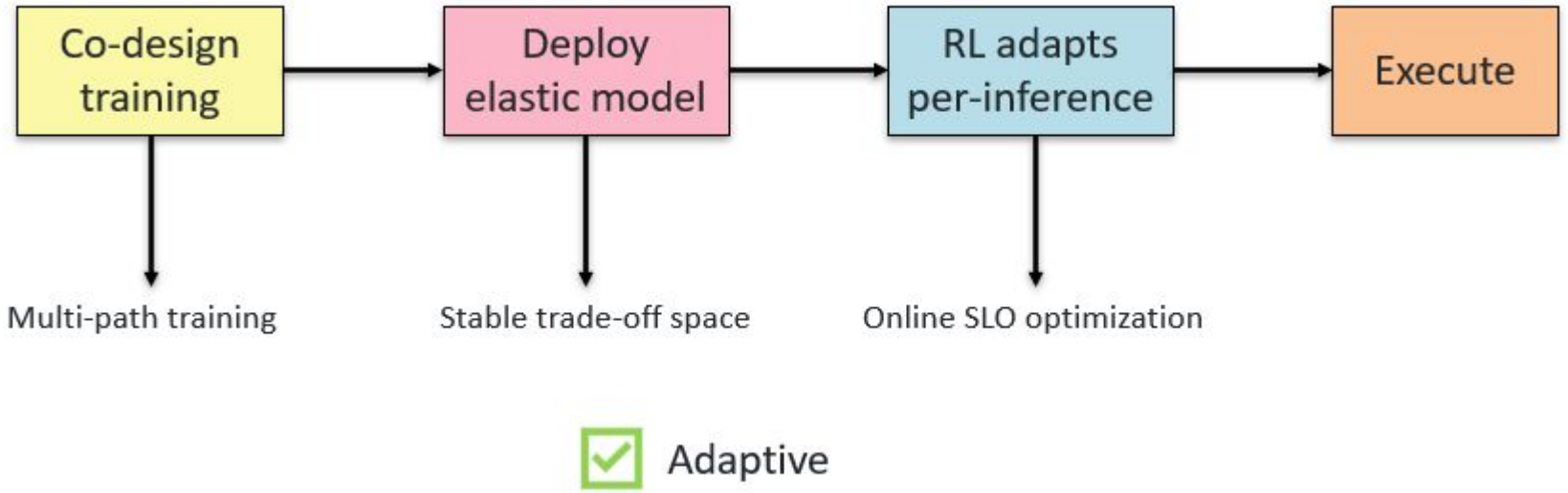
- **Our Approach:**



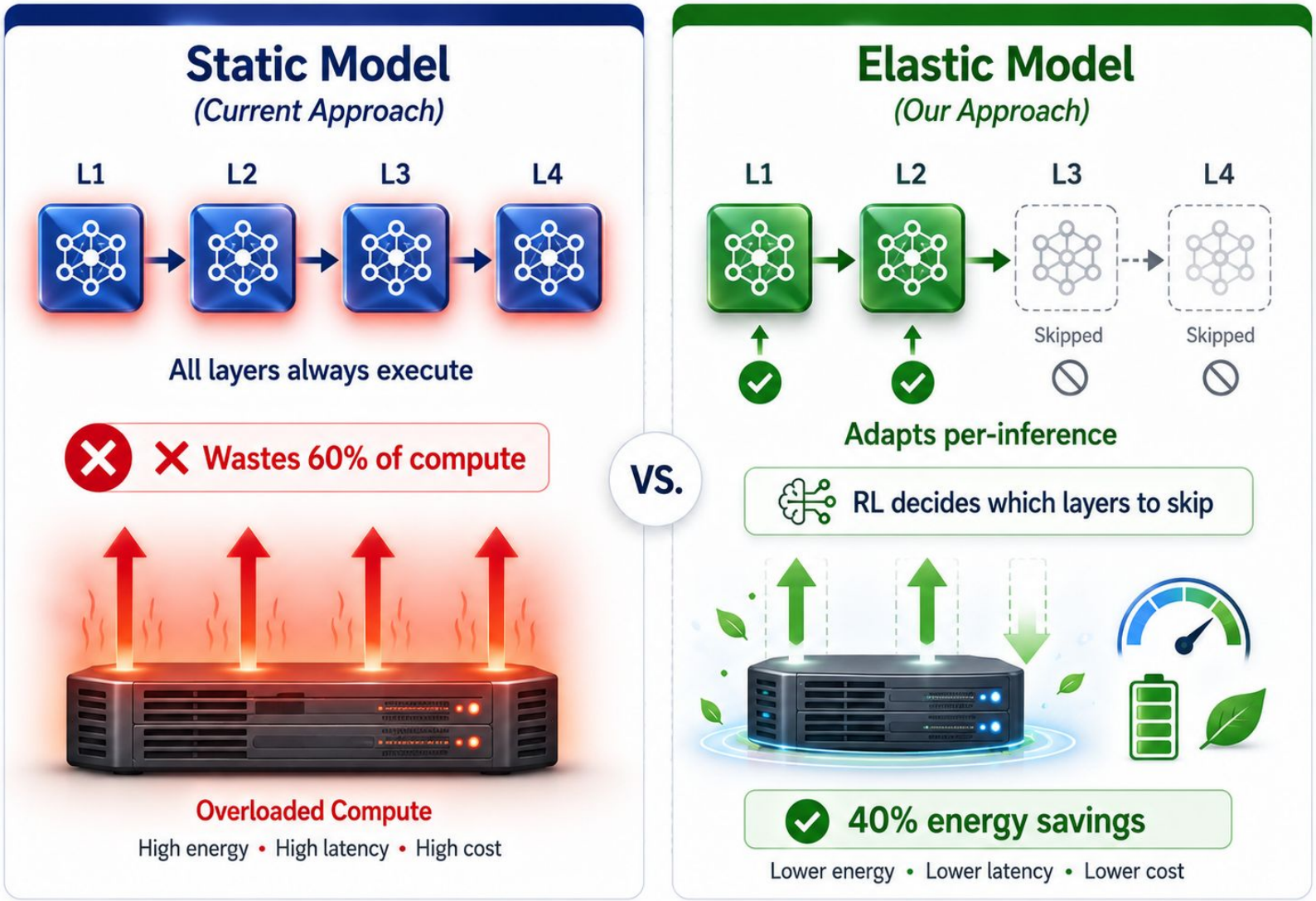
Challenge #3:
Elastic Model and Runtime Co-Design

- **Prior Work:**

- **Our Approach:**



Challenge #3:
Elastic Model and Runtime Co-Design



Elastic inference = Smart execution, Maximum efficiency

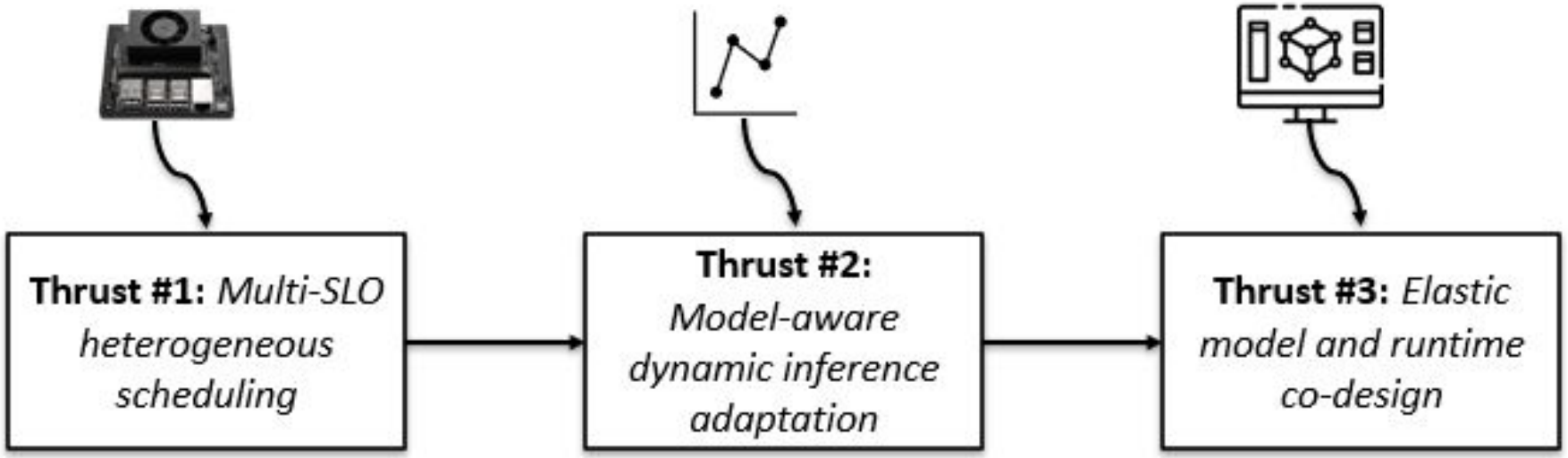
Problem Statement

- How can we design a unified edge AI framework that integrates elastic model architectures with intelligent runtime adaptation to simultaneously achieve multi-SLO satisfaction, energy efficiency, and resilience in heterogeneous, resource-constrained environments?

Agenda

- Motivation and Research Challenges.
- **Proposed Approach**
  1. Research Part #1 – Multi-SLO heterogeneous scheduling
  2. Research Part #2 – Model-aware dynamic inference adaptation
  3. Research Part #3 – Elastic model and runtime co-design
  4. Evaluation Plan
- Timeline, Summary, and Contributions

Proposed Research Overview



Research Thrust #1:

Multi-SLO heterogeneous scheduling

Edge AI "Scheduling" Challenges

- **Simultaneously Meeting Multiple SLOs (Service Level Objectives) in Edge AI Scheduling**
  - Edge devices are resource-limited and include heterogeneous AI accelerators (e.g., EdgeGPUs, EdgeTPUs)
  - Scheduling becomes more complex when executing multiple concurrent AI tasks
- Edge AI tasks must satisfy multiple SLOs:
  - Accuracy
  - Throughput
  - Deadline (Latency)

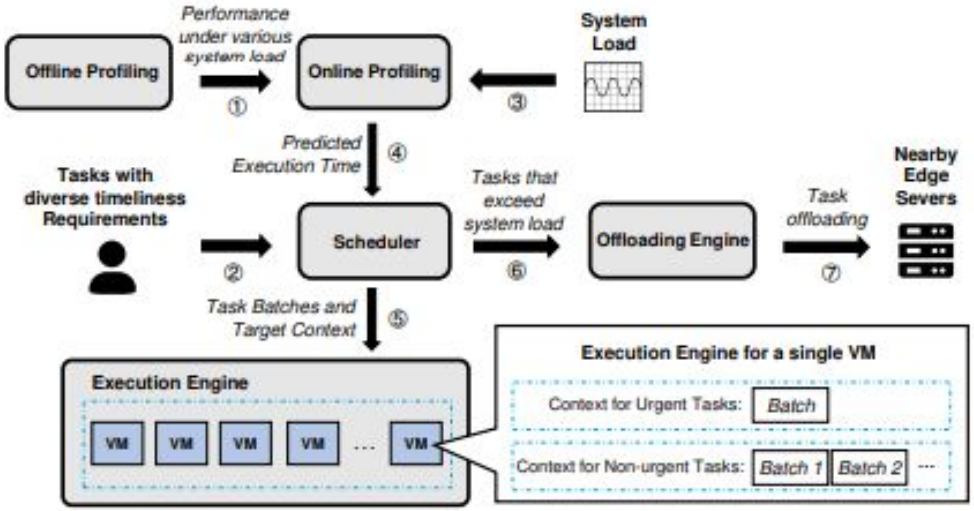
Research Thrust #1

- Challenge
  - Simultaneously Meeting Multiple SLOs (Service Level Objectives) in Edge AI Scheduling

- Research goal
  - Establish the foundational system-level orchestration layer that intelligently coordinates heterogeneous accelerators and static models to maximize multi-SLO satisfaction, while revealing the fundamental efficiency ceiling imposed by the black-box model paradigm.

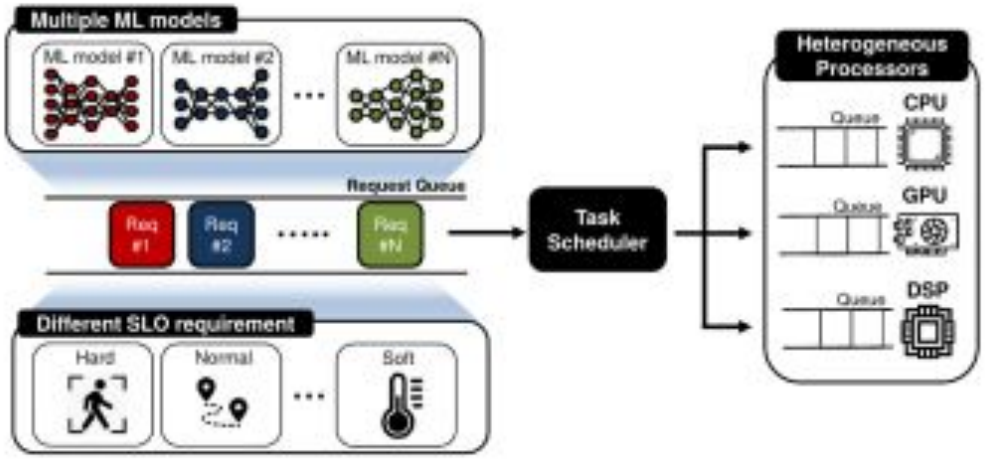
Research Thrust #1 -- Related Work

- Park, Seonyeong, et al. (IEEE Access '21)
- Kalmia is a scheduling framework for edge servers that manages heterogeneous DNN inference tasks while meeting QoS requirements by co-locating different types of workloads (latency-critical and best-effort) on GPUs and CPUs.



Research Thrust #1 -- Related Work

- Seo, Wonik, et al. (ACM TACO ’21)
- This paper presents an SLO-aware scheduler for DNN inference on heterogeneous edge platforms (CPUs, GPUs, and accelerators) that dynamically assigns inference tasks to different processors to meet service level objectives (SLOs). The scheduler uses runtime profiling and prediction models to estimate execution times and makes scheduling decisions that maximize resource utilization while satisfying latency requirements.



Research Thrust #1 -- Related Work

- Model Compression (Han et al., Lee et al.): Reduces size, but often degrades performance under strict SLOs (e.g., accuracy/latency).

- Distributed Inference (Xie et al., Zhang et al.): Distributes workloads across devices, but high variability due to heterogeneous hardware.

- Adaptive Inferencing (Wang et al.): Dynamically compresses, but struggles to consistently meet multiple SLOs (e.g., accuracy drops).

Research Thrust #1 -- Related Work

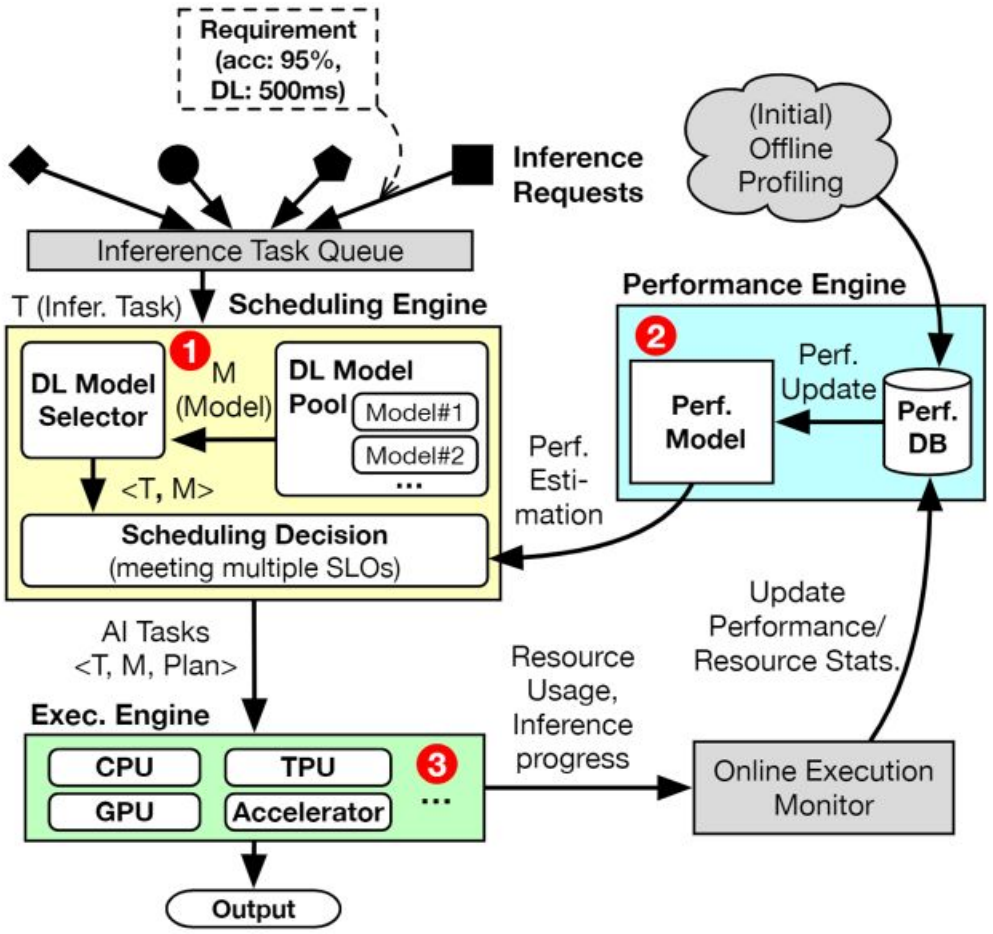
- Model Compression (Han et al., Lee et al.): Reduces size, but often degrades performance under strict SLOs (e.g., accuracy/latency).

- Distributed Inference (Xie et al., Zhang et al.): Distributes workloads across devices, but high variability due to heterogeneous hardware.

- Adaptive Inferencing (Wang et al.): Dynamically compresses, but struggles to consistently meet multiple SLOs (e.g., accuracy drops).

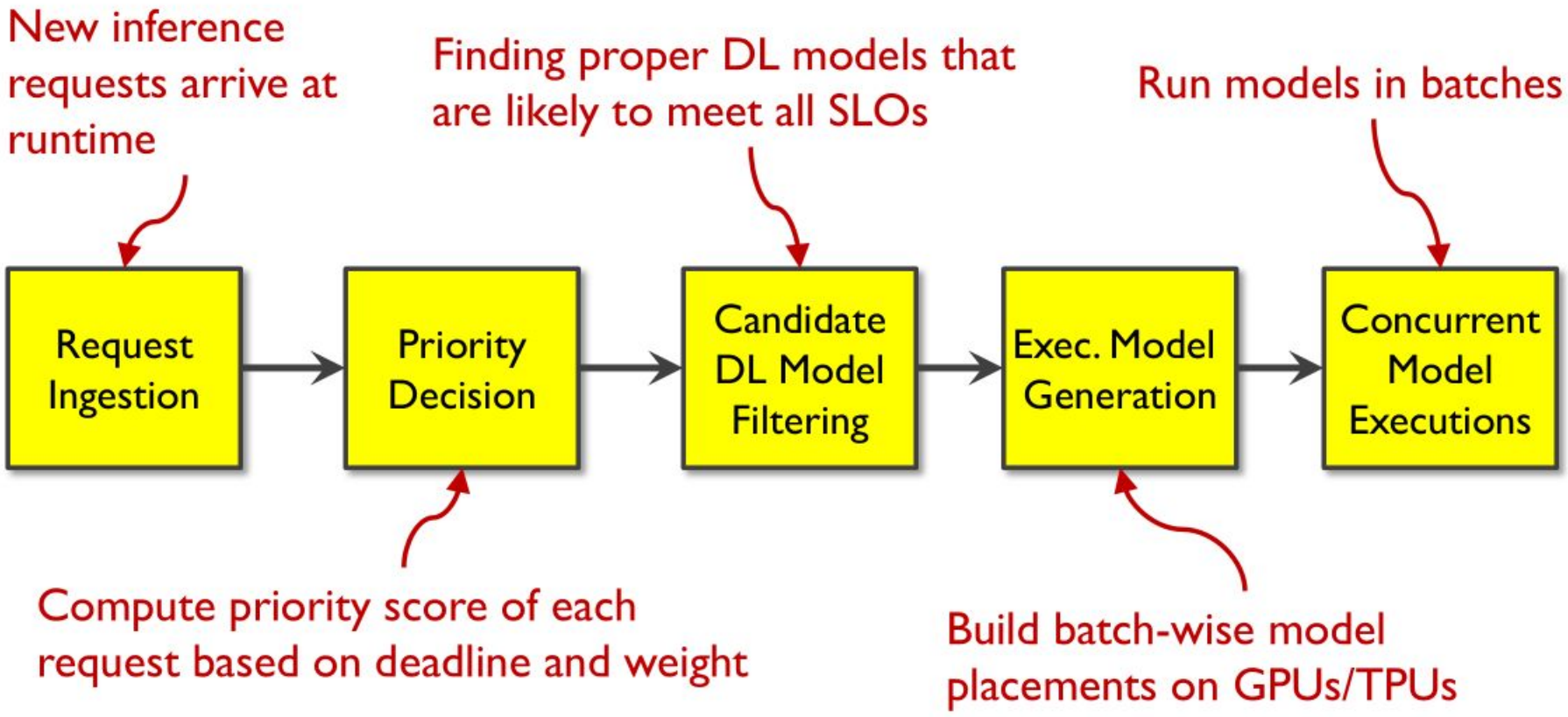
Research Thrust #1

1. Scheduling Engine
   - Select best models to meet SLOs
2. Performance Engine
   - Profiles and estimate model perf.
3. Execution Engine
   - Runs models on AI accelerators



Research Thrust #1

- Scheduling Overview



Research Thrust #1

- User Requests and Priority
  - Convergo handles multiple concurrent inference tasks with dynamic priority scheduling.
- Each user request (R_i) includes multiple SLOs:
  - Throughput (Th_i), Accuracy (Acc_i), Deadline (D_i)
- All requests are placed in a **priority queue**

Deadline Current Time Arrival Time

$$Priority_i = \frac{D_i - (t_{curr} - t_{arr,R_i})}{w_i}$$

w_i ← User assigned weight (1 -- 10)

- Priority scores are dynamically updates over time

Research Thrust #1

- After prioritization, Convergo checks 3 constraints to find candidate DL models.
  - Constraint #1: Throughput $\sum_{j=1}^{n} \mathcal{M}_j(Th) \times t_j \geq Th_i,$
  - Constraint #2: Accuracy $\frac{\sum_{j=1}^{n} \mathcal{M}_j(Th) \times t_j \times \mathcal{M}_j(Acc)}{\sum_{j=1}^{n} \mathcal{M}_j(Th)} \geq Acc_i,$
  - Constraint #3: Deadline $\sum_{j=1}^{n} t_j \leq D_i.$
- Among the candidates, choose those with minimal total execution time (to maximize slack).
- Generate exec. plan from the selected models.

Research Thrust #1

- Edge Platform: Nvidia Jetson Xavier
  - 6-core CPUs, 8G RAM, Volta GPUs
- Augmented with multiple TPU acc.
  - 4TOPS @ 2W
- AI Applications and Models

Jetson Xavier

Coral
TPU Accelerator

3 Diff Apps

AI Application	Edge Acceler	Pre-trained DL Models	Size (MB)	Num. Layers
Image Classification	GPU	EfficientNet [36]	5.4	5
		Inception-v1 [34]	6.4	22
		MobileNet-v2 [29]	3.4	20
		SqueezeNet [16]	1.3	15
	TPU	EfficientNet	6.8	7
		Inception-v1	7.0	22
		Inception-v3 [35]	12.0	48
		MobileNet-v1 [15]	1.6	28
Object Detection	GPU	EfficientNet	4.5	5
		MobileNet-v2	6.5	20
		YOLOX [12]	5.4	11
	TPU	EfficientNet	7.6	18
		MobileNet-v1	7.0	28
		MobileNet-v2	6.6	20
Pose Estimation	GPU	MobileNet-v1 [15]	8.2	28
		MoveNet [3]	5.6	25
		PoseNet [25]	4.3	23
	TPU	MoveNet	7.5	25
		PoseNet-MobileNet [25]	1.5	23
		PoseNet-ResNet [25]	24.4	18

GPU Models on TensorFlow

TPU Models on TFLite

Research Thrust #1

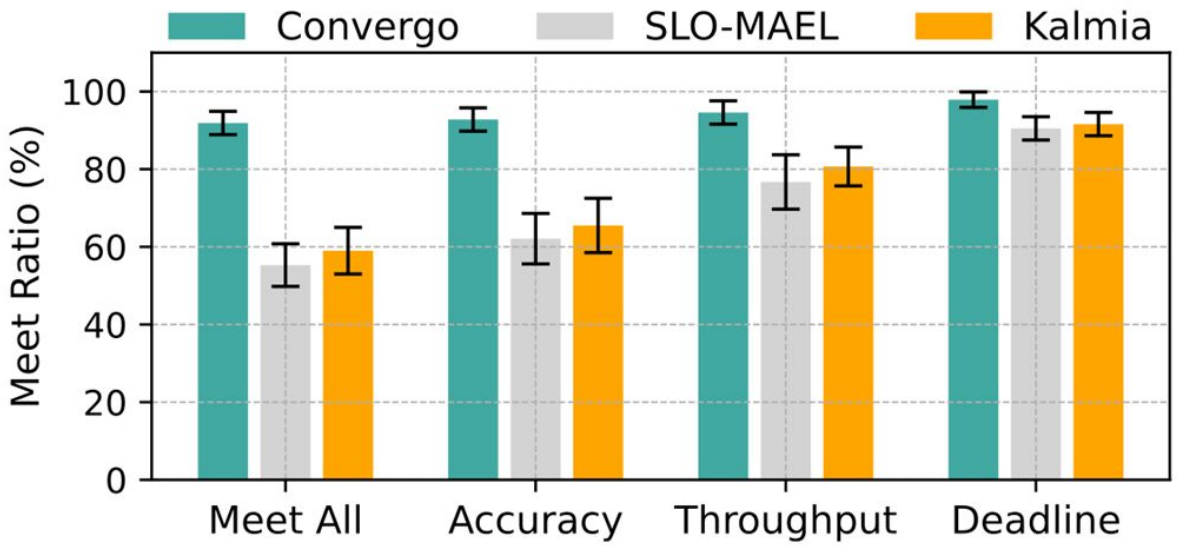
- Workload Generation
  - Mixed AI tasks: IC, OD, PE
  - 500 reqs per workload, generated using Poisson distribution
  - Each req includes unique SLOs: accuracy, throughput, and deadline
  - Evaluated over 10 workload sets to reduce bias and ensure robustness

- Baselines
  - SLO-MAEL (ACM TACO'21): Prioritizes based on estimated latency and risk of SLO violation; uses preemptive execution
  - Kalmia (INFOCOM'24): QoS-aware scheduler for heterogeneous DNNs; combines preemptive and context-aware policies

Research Thrust #1 – Preliminary Results

- Measured both “Meet All” and individual SLOs

- Meet All: satisfying all three SLOs simultaneously

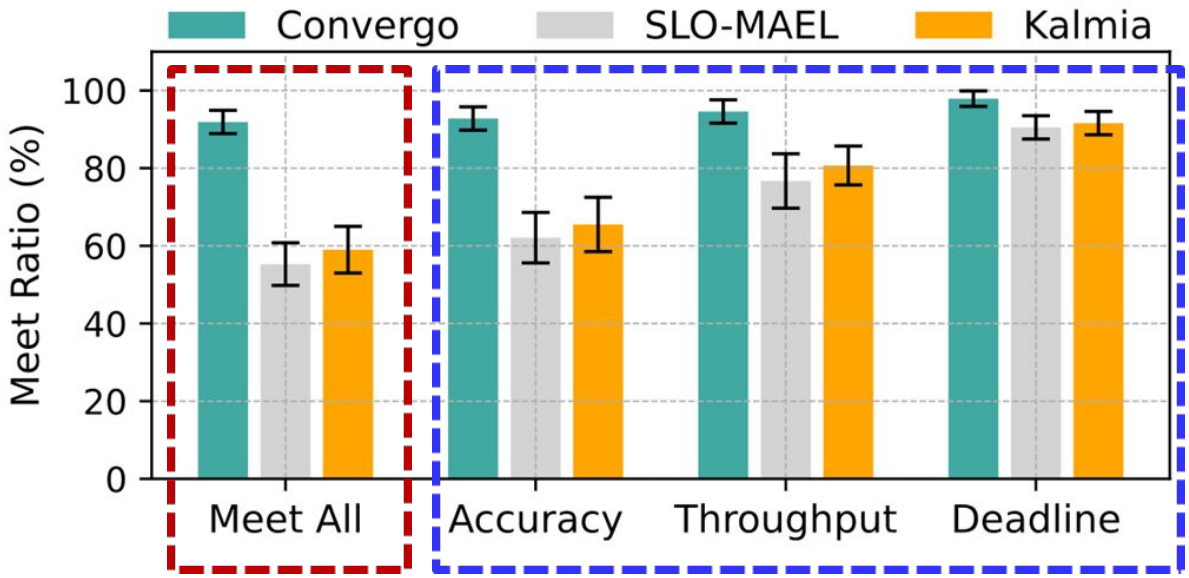


Research Thrust #1 – Preliminary Results

- Measured both “Meet All” and individual SLOs
- Meet All: satisfying all three SLOs simultaneously

Meet All:
- Convergo: over 95%
- Baselines: less than 60%

Individual SLOs:
- Convergo: 93% (acc), 95% (th), 98% (DL)
- Baselines: less than 70% (acc), less than 80% (th)

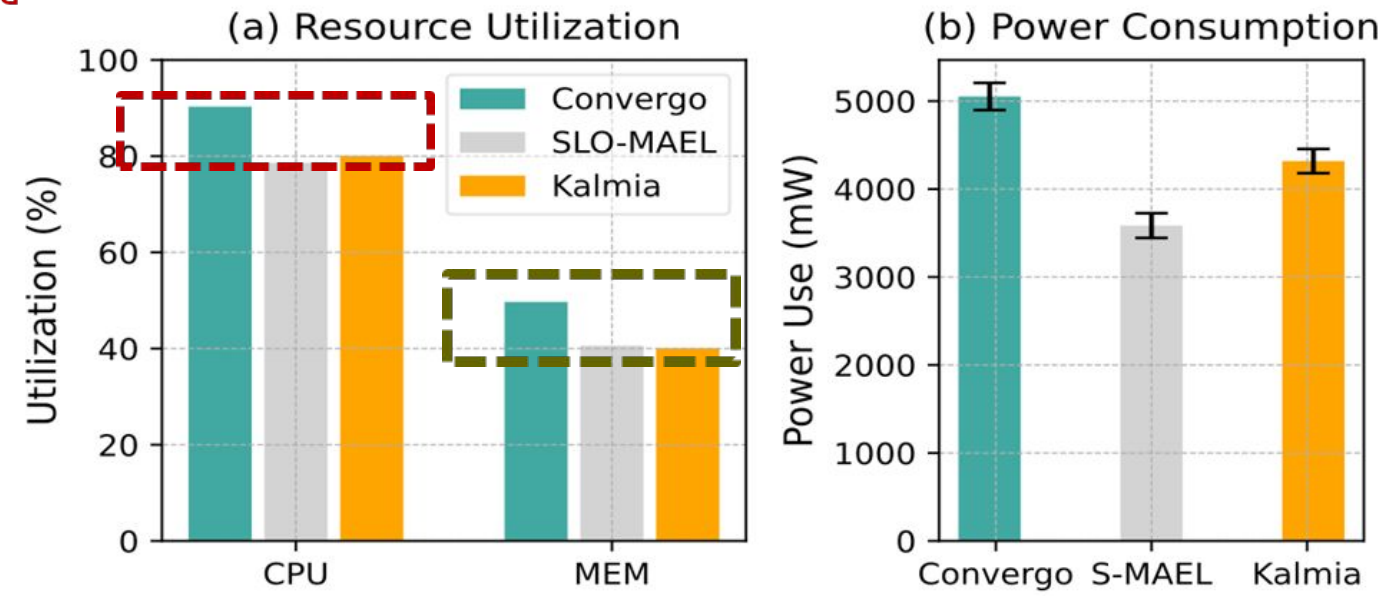


Research Thrust #1 – Preliminary Results

- Resource Utilization

CPU util increased by 12%

MEM util increased by 10%



Research Thrust #1 – Preliminary Results

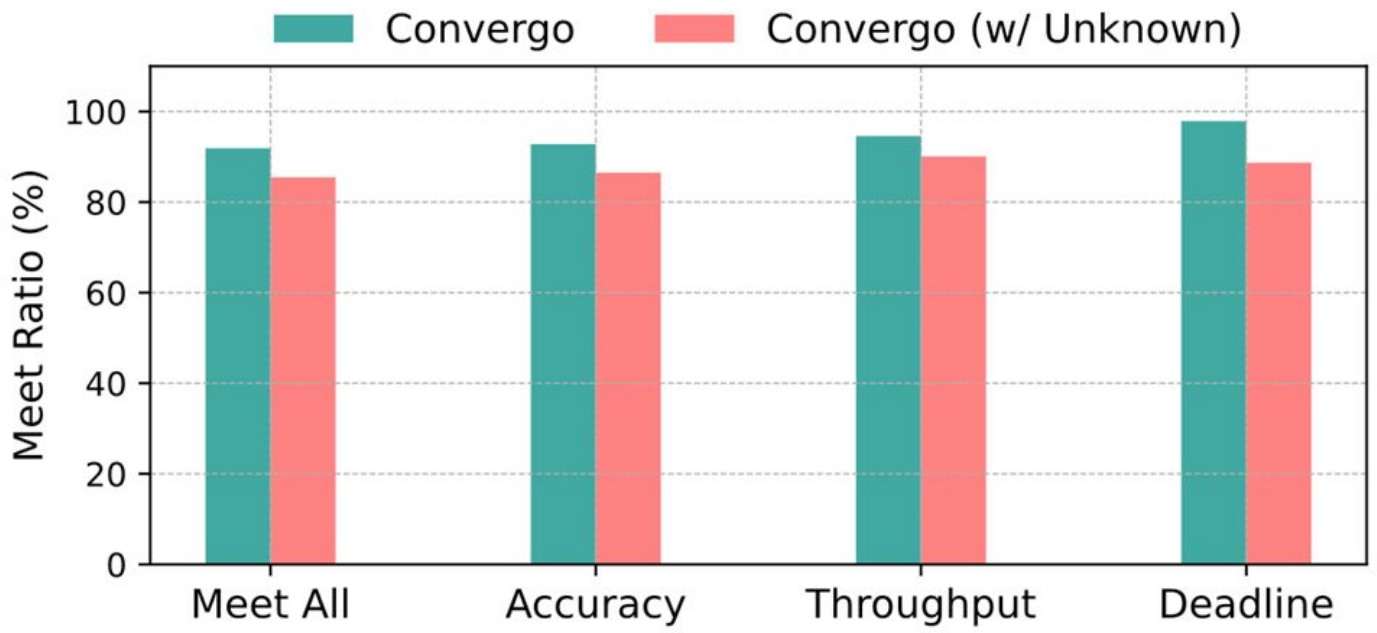
- Covergo Scheduler Overhead

	Avg	Std Dev
Extra CPU Consme..	6.3%	1.6
Extra MEM Consume.	4.9%	1.1
Extra Power Consume.	527mW	45

Still, less than 10% of dev power.

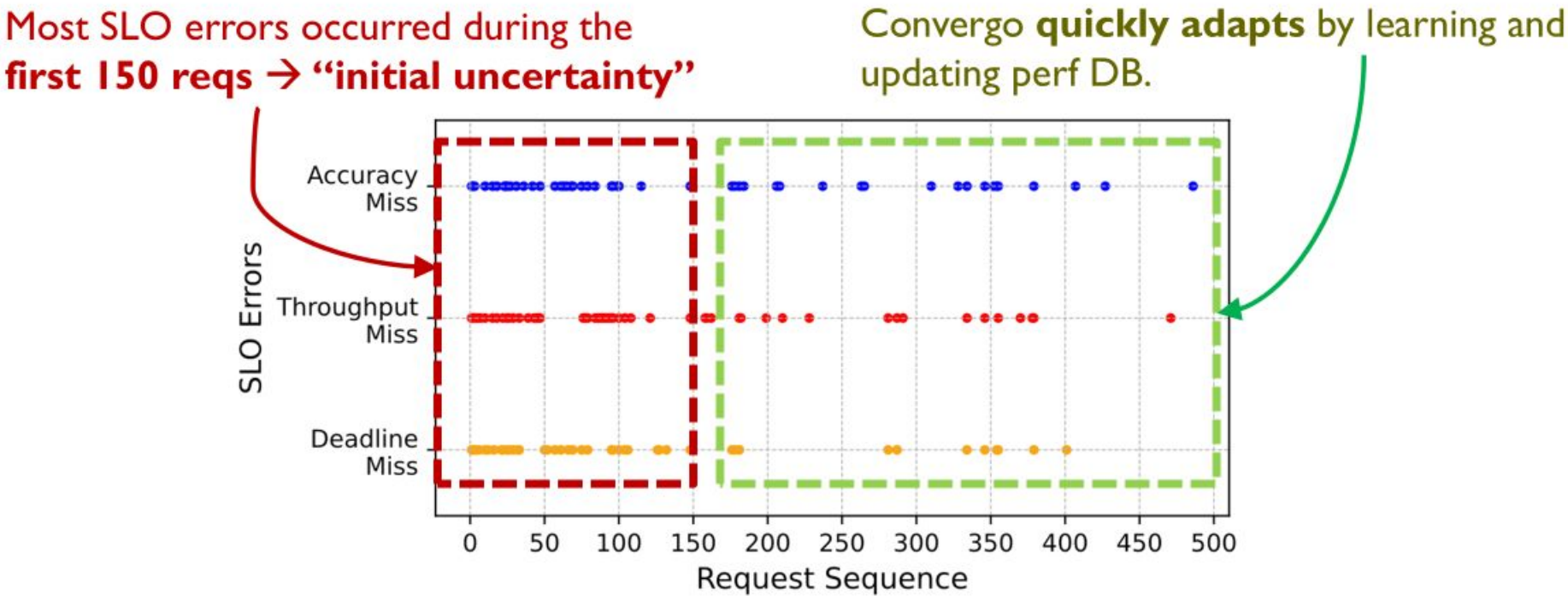
Research Thrust #1 – Preliminary Results

- Performance with ‘unknown’ Models
  - 30% of models were previously unknown (i.e., unprofiled).



Research Thrust #1 – Preliminary Results

- Performance with 'unknown' Models
  - 30% of models were previously unknown (i.e., unprofiled).



Research Thrust #2:

Model-aware dynamic inference adaptation

Research Thrust #2

- Challenge

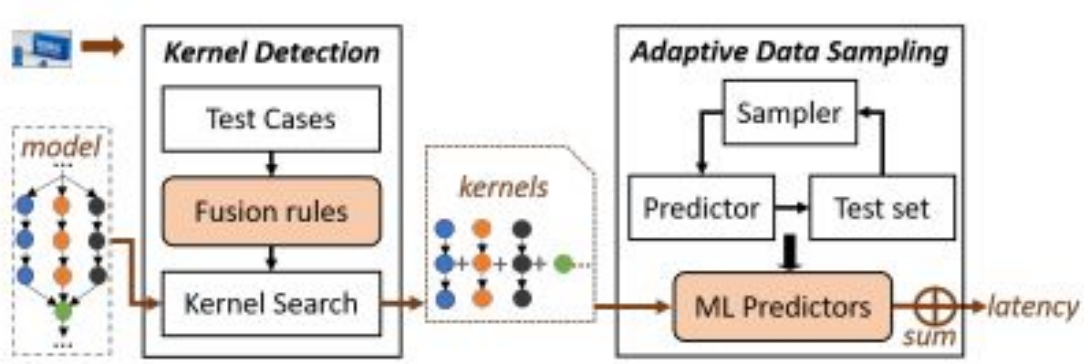
  The massive configuration space makes real-time optimal execution path selection computationally infeasible without accurate prediction models.

- Research goal

  Develop skip-aware models and lightweight prediction mechanisms that expose a quantifiable accuracy–performance trade-off space for safe and efficient runtime adaptation.

Research Thrust #2 -- Related Work

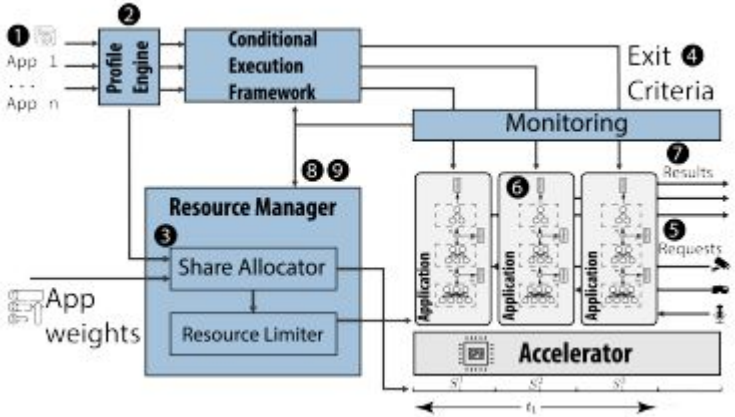
- Zhang, Li Lyna, et al. (MobiSys'21)
  - Proposes a learning-based framework to predict DNN inference latency on diverse edge devices.
  - Builds a latency predictor using operator-level profiling and regression modeling.



- Limitations
  - Predicts latency only; does not model accuracy, energy consumption, or multi-objective trade-offs needed for SLO satisfaction.
  - q Requires extensive offline kernel profiling for each new device; adaptation overhead limits real-time reconfiguration scenarios.
  - q Static prediction model cannot adapt to dynamic system conditions

Research Thrust #2 -- Related Work

- Liang, Qianlin, et al. (IoTDI'23)
  - Adaptive model-serving system for multi-tenant edge environments that dynamically adjusts batch size and power modes.
  - Uses profiling-based performance models to predict latency and resource usage for different configurations.



- Limitations
  - Treats models as static black-box variants (no internal structural elasticity).
  - Adaptation occurs at the model granularity, not layer/block level.
  - Does not provide predictable accuracy–performance degradation modeling.

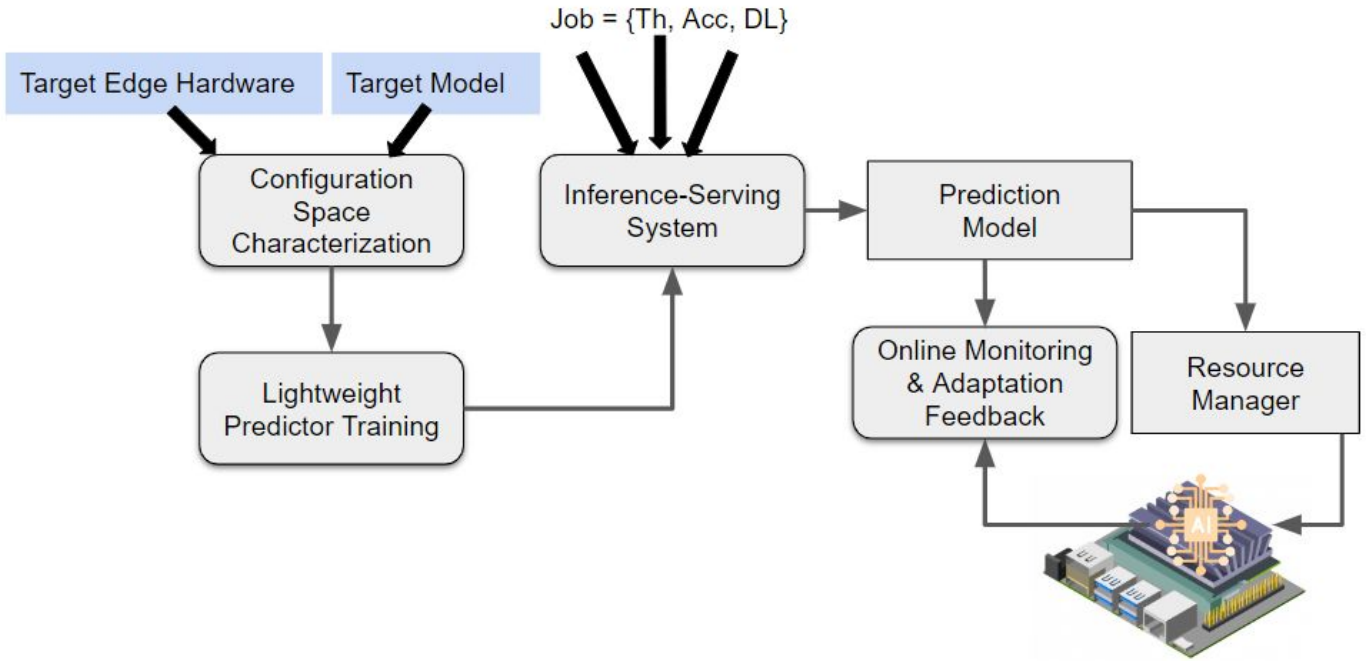
Research Thrust #2

- Offline model

  Enumerates a set of allowable skip configurations

- Online stage

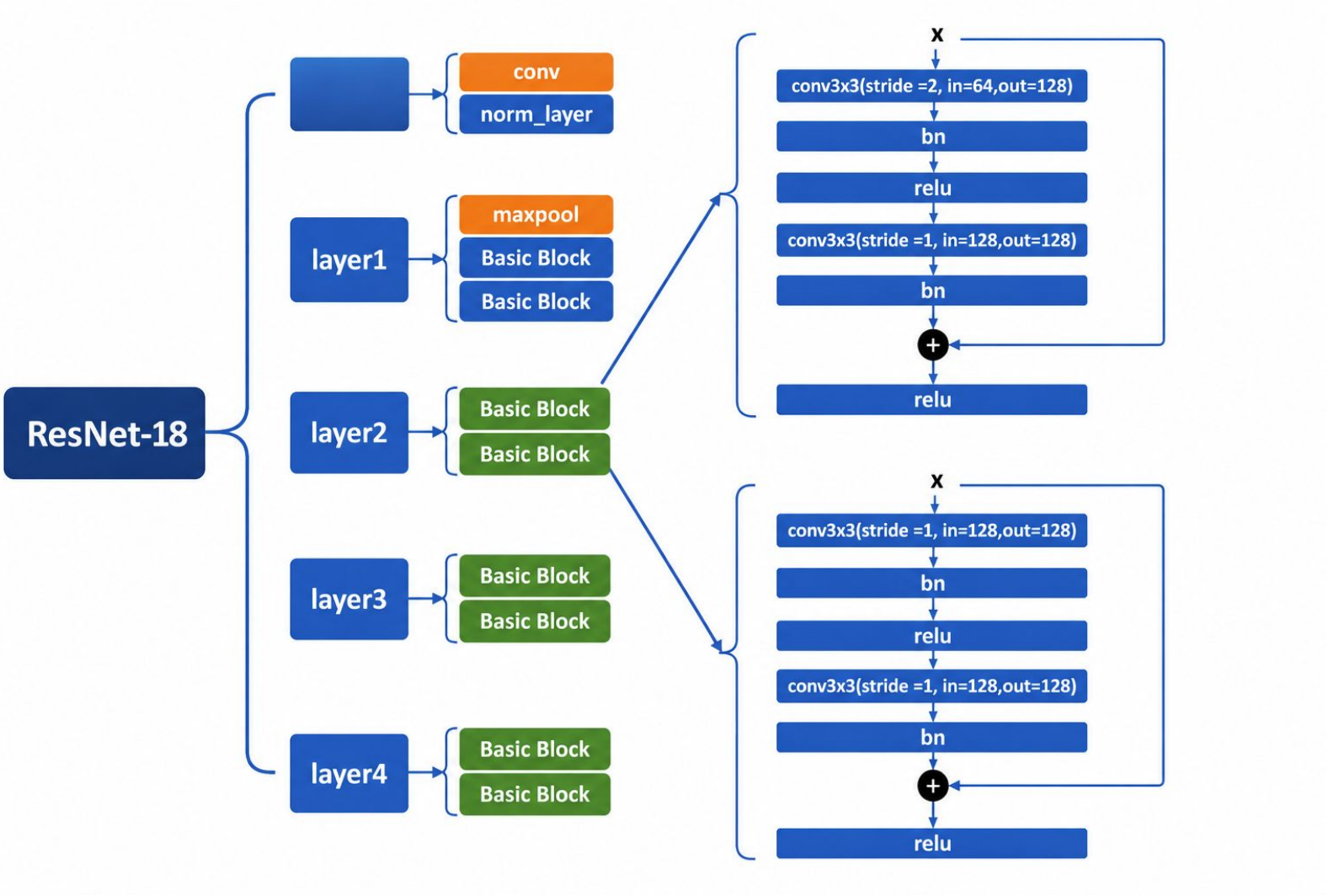
  Select a suitable skip configuration



Research Thrust #2

Accuracy Prediction Model: predict the inference accuracy (e.g., top-1 or top-5) for a given model-level skip-configuration.

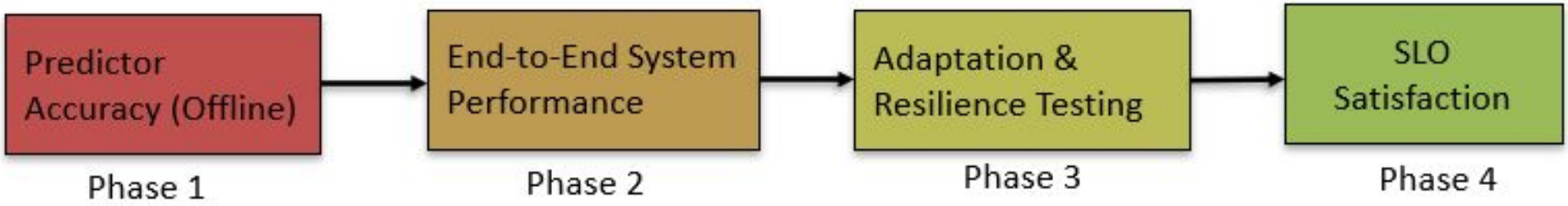
Research Thrust #2



Evaluation Plan

- RQ1: How accurate are our lightweight prediction models (latency + accuracy)?
- RQ2: Can dynamic layer-skipping meet diverse SLOs while reducing energy?
- RQ3: What is the overhead of runtime decision-making?
- RQ4: How does the system adapt to workload variations?

Evaluation Plan



Evaluation Plan

- Hardware Platforms:

Device	CPU	GPU	Memory	Power	Price
Jetson Orin Nano	6-core Arm Cortex-A78AE	1024-core NVIDIA Ampere	8GB	7–15W	$499
Jetson AGX Xavier	8-core Carmel ARM	512-core Volta + Tensor	32GB	10–30W	$699
GeForce RTX 2080 Ti	8-core 64-bit CPU	4352 CUDA cores	16GB	250W	$999
RTX A5000	48-core 64-bit CPU	8192 CUDA cores	196GB	230W	$2500

Evaluation Plan

- Models:

ResNet-18, ResNet-34, ResNet-50, ResNet-74, ResNet-101, ResNet-152

- Dataset: CIFAR-10 and CIFAR-100
- Layer-skip configurations: 4,095 paths per model (ResNet-34 example)
- Batch sizes: {1, 2, 4, 8, 16, 20}
- Power modes: 8 combinations (4 modes × jetson_clocks on/off)
- Total: About 160,000 unique configurations per model

Evaluation Plan

Baselines:

- B1: Static Full-Model Execution
  - Always execute complete model at maximum power
  - Represents "no adaptation" approach
- B2: nn-Meter (MobiSys'21)
  - Uses their latency predictor
  - No accuracy prediction or dynamic skipping
- B3: Dělen (IoTDI'23)
  - System-level adaptation (batch + power only)
  - No model-internal skipping
- B4: Fixed Skip Configuration & Random Skip Policy
  - Pre-selected "best average" skip pattern, no per-inference adaptation
  - Randomly select skip configuration, control for predictor effectiveness

Evaluation Plan

Comparison Matrix:

Approach	Model-Internal	SLO-Satisfaction	Online	Energy-Aware
Static Full	✗	✗	✗	✗
Dělen	✗	Partial	✓	Partial
nn-Meter	✗	✗	✗	✗
Fixed Skip	✓	✗	✗	✗
Random	✓	✗	✓	✗
Thrust 2 (Ours)	✓	✓	✓	✓

Evaluation Plan

Expected Results:

- Prediction Accuracy
- SLO Satisfaction
- Energy Savings
- Overhead
  - Prediction time
  - Total overhead

Research Thrust #3:

Elastic model and runtime co-design

Research Thrust #3

- Challenge

  Static models exhibit unpredictable accuracy drops when dynamically reconfigured at runtime.

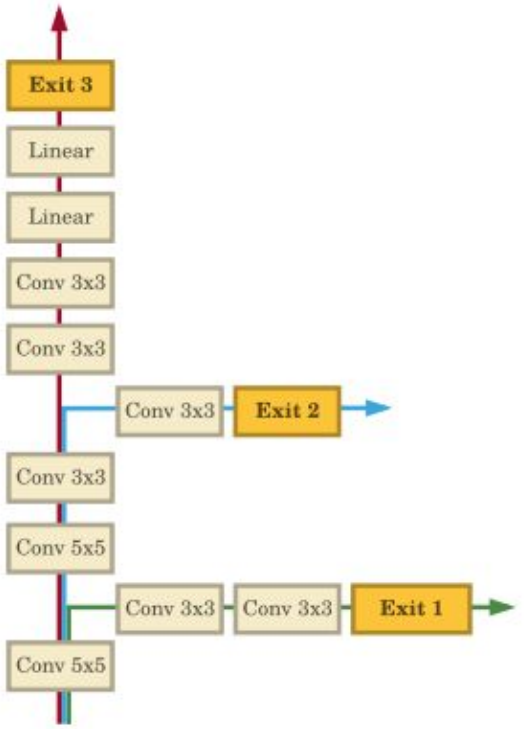
- Research goal

  Co-design elastic models and RL-based runtime policies for stable, autonomous adaptation.

Research Thrust #3 -- Related Work

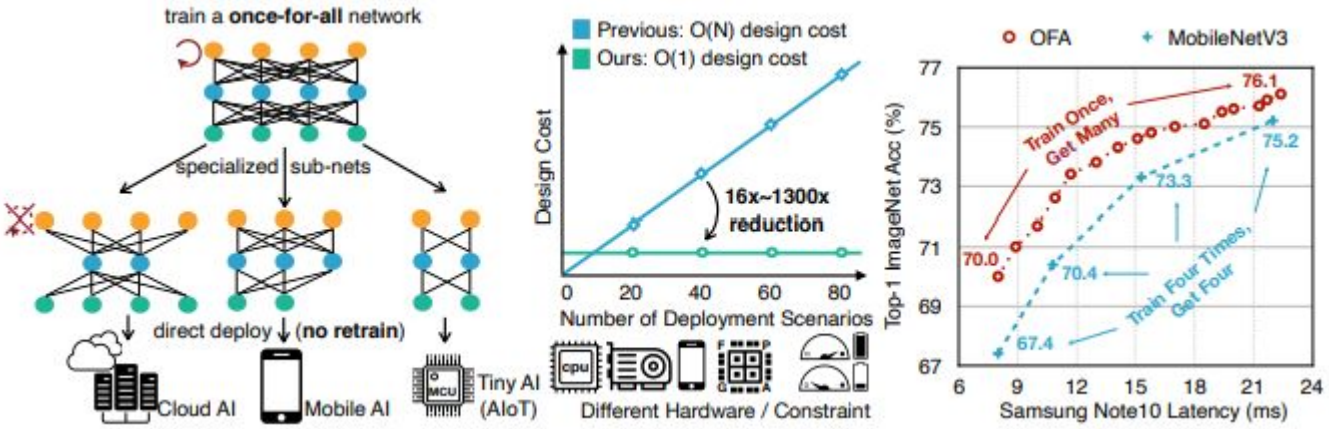
- Teerapittayanon, S, et al. (ICPR'16)
  - Introduced early-exit branches inside deep neural networks to reduce average inference latency.
  - Enables inference to terminate early
  - Reduces computation by allowing "easy" inputs

- Limitations
  - Early-exit decisions rely solely on confidence threshold heuristics, not system-level SLO awareness.
  - No coordination with hardware heterogeneity (CPU/GPU/TPU).
  - Does not optimize for energy or multi-SLO constraints.



Research Thrust #3 -- Related Work

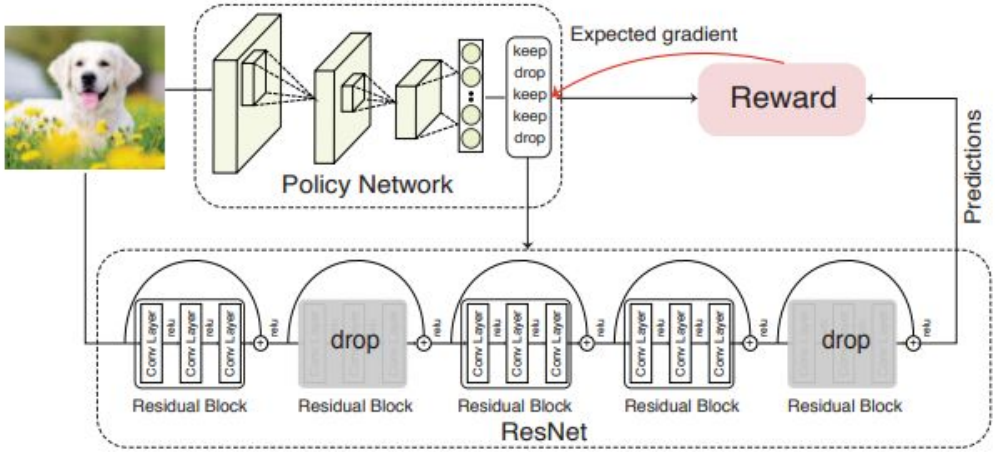
- Cai, Han, et al. (ICLR'20)
  - Proposes training a single over-parameterized super-network that supports many sub-networks with different depths, widths, kernel sizes, and resolutions.



- Limitations
  - Specialization is performed offline, not dynamically at runtime.
  - Does not consider multi-SLO trade-offs during inference serving.
  - No online adaptation to workload variation or data drift.
  - Lacks runtime policy optimization for real-time control.

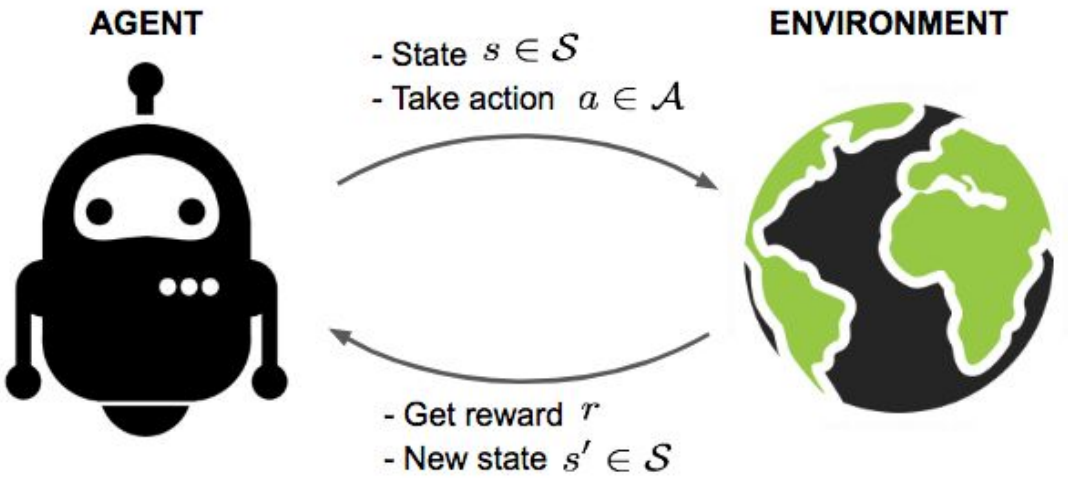
Research Thrust #3 -- Related Work

- Wu, Zuxuan, et al. (CVPR'18)
  - Proposes dynamically dropping residual blocks in ResNet during inference based on input difficulty.
  - Uses a reinforcement learning agent to learn block execution policies.
- Limitations
  - Policy network is trained offline with fixed reward function; cannot adapt to changing runtime SLO requirements.
  - Focuses solely on latency reduction, ignoring energy consumption and multi-objective optimization



Research Thrust #3

- **Reinforcement learning**
  - A machine learning training method based on rewarding desired behaviors and punishing undesired ones.

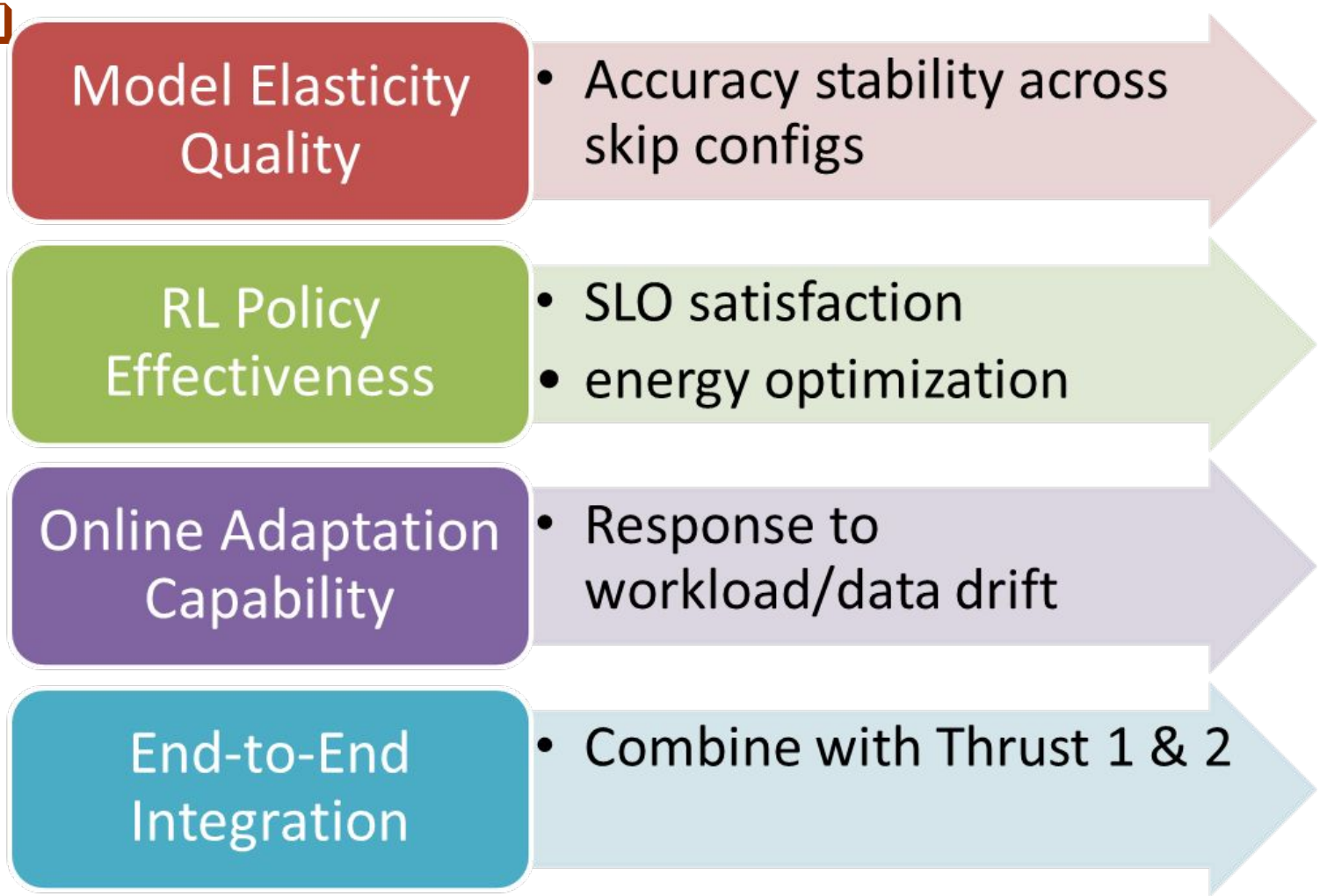


Evaluation Plan

Research Questions to Answer:

- RQ1: Can our elastic training produce models with stable accuracy degradation?
- RQ2: Does online RL policy outperform offline/fixed policies?
- RQ3: Can the system autonomously adapt to changing SLO requirements?
- RQ4: How does co-design compare to decoupled approaches?

Evaluation Plan



Evaluation Plan

Baselines:

- B1: Static Full Execution(With Thrust 1 Scheduler)
  - Use Convergo for model selection
  - Always execute full model
  - No dynamic adaptation
- B2: BranchyNet (ICPR'16)
  - Early-exit approach + Fixed confidence thresholds
  - No SLO awareness
- B3: Once-for-All + Offline Selection (ICLR'20)
  - Select sub-network offline based on average SLO
  - Elastic super-network, but no runtime adaptation
- B4: BlockDrop + Fixed Policy (CVPR'18)
  - RL-trained policy (offline)
  - Input-dependent but not SLO-aware

Evaluation Plan

Comparison Matrix:

Approach	Elastic Training	SLO-Aware	Online Adapt	Multi-Obj
Static + T1	✗	✓	✗	✓
BranchyNet	Partial	✗	✗	✗
OFA Offline	✓	Partial	✗	✗
BlockDrop	✓	✗	Partial	✗
T3 (Ours)	✓	✓	✓	✓

Evaluation Plan

Expected Results:

- Elastic Model Stability
- RL Policy vs. Fixed
- SLO-Satisfaction (with Energy)
- Adaptation Capability

Agenda

- Motivation and Research Challenges.
- Proposed Approach
  1. Research Part #1 – Multi-SLO heterogeneous scheduling
  2. Research Part #2 – Model-aware dynamic inference adaptation
  3. Research Part #3 – Elastic model and runtime co-design
  4. Evaluation Plan
- **Timeline, Summary, and Contributions**

Timeline

	Semester:	1	2	3	4	5
Research	Thrust 1	■	■			
	Thrust 2		■	■	■	
	Thrust 3				■	■
Paper	Paper Submission		■			■
Thesis	Thesis Writing & Defense					■

Table 2: **Project Timeline**

Summary and Contributions

- Summary
  - This project aims to design an energy-efficient and multi-SLO-aware edge AI inference system through elastic model-runtime co-design on heterogeneous accelerators.
- Research Contributions
  1) Designing multi-SLO heterogeneous scheduler for concurrent accuracy, throughput, and deadline satisfaction
  2) Building lightweight prediction models for latency and accuracy across massive configuration spaces
  3) Developing reconfigurable inference engine with dynamic layer-bypassing capabilities
  4) Creating co-design methodology for intrinsically elastic neural network architectures

Thank you!